


# Lab 1: Modelling of the Motor System

MTE 484: Digital Control Systems

Instructor Sergio Livera

Department of Mechatronics and Mechanical Engineering

**Levi Rogalla**
**Alex Veric**

November 2, 2026

We acknowledge and promise that:

(a) We are the sole authors of this lab report and associated simulation files/code.

(b) This work represents our original work.

(c) We have not shared detailed analysis or detailed design results, computer code, or Simulink diagrams with any other student.

(d) We have not obtained or looked at lab reports from any other current or former student of ECE 481/ ECE 484 / MTE 484, and we have not let any other student access any part of our lab work.

(e) We have completely and unambiguously acknowledged and referenced all persons and aids used to help us with our work.

- ✓ **Student 1 Name:** Levi Rogalla
- ✓ **Student 1 Signature*:** 
- ✓ **Student 2 Name:** Alex Verić
- ✓ **Student 2 Signature*:** 
- ✓ **Lab Group #:** 57
- ✓ **Station #:** 3

*(*) We require an actual signature: a typed name is not an acceptable substitute.*

# 1 Introduction

This lab introduces the motor and gear mechanism which are to be used for the 'ball balancing on beam' project, which must be properly characterized in order to facilitate the completion of the project. This includes experimentally determining values for 'stiction' and the transfer function of the motor. These activities were performed during the span of the lab session, and their procedures and results are described in detail in the following report.

# 2 Question C

By this point in the lab, the set up was already complete. The Arduino was functioning as intended, and with the sample code provided, it was possible to both send voltage values to the motor and read sensor data, which included the motor rotational position and the ball's position on the beam. These values were printed out on the serial monitor, allowing them to be recorded.

The values of the motor position being output needed to be converted to radians in order to be useful. The motor position sensor values could be linearized and converted into radians using the following formula:

$$p_{radians} = m * p_{sensor} + b \tag{1}$$

Where the $m$ and $b$ values must be determined through a system of equations, using experimental values of $p_{rad}$ and $p_{sensor}$ to do so.

The position of the motor was manually changed to three points: 0, $\frac{\pi}{4}$, and $-\frac{\pi}{4}$ radians. Two points are sufficient to determine a linear relationship, while the third point provides a check on the result. The values printed into the serial monitor were read at each point, providing values of $p_{rad}$ in radians and $p_{sensor}$ in ADC counts. The two measurements used to calculate the line were $p_{sensor} = 4890.47$ at $p_{rad} = 0$ rad and $p_{sensor} = 7075.45$ at $p_{rad} = -\frac{\pi}{4}$ rad:

$$\begin{bmatrix} 4890.47 & 1 \\ 7075.45 & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ -\frac{\pi}{4} \end{bmatrix} \tag{2}$$

$$m = -0.000359453251 \text{ rad/count}, \qquad b = 1.75789534 \text{ rad} \tag{3}$$

The resulting scaling was implemented in software as

$$\theta = -0.000359453251\, p_{sensor} + 1.75789534.$$

The third measured angle was used as a consistency check.

# 3 Question D

The term 'stiction' describes the concept of static friction that exists in motors before they start moving. This creates a range of voltage values that, when sent to the motor, will not move the motor at all. This impacts the accuracy of the mechanism, especially when considering the fine movements required to balance a ball on a beam. This must be counteracted by creating a 'deadzone' at a certain range; for example, if the deadzone range is determined to be -0.2 to 0.1 volts, we simply add 0.1 to positive voltages and -0.2 to negative voltages.

The values of voltage that determine the deadzone range must be experimentally determined. Additionally, it is important to determine them in both directions, as the stiction in motors is not often symmetrical. In order to complete this task, the team created a function that gradually

increases the voltage sent to the motor and reads back the angle in each iteration, once the angle changed by more than 0.06 radians, the corresponding voltage was printed.

The values for stiction have been determined for both directions, differentiated by whether the voltage values were increasing or decreasing:

$$v_+ = +0.120 \text{ V}, \qquad v_- = -0.101 \text{ V} \tag{4}$$

Here, $v_+$ is the offset applied when the controller requests a positive voltage and $v_-$ is the offset applied when it requests a negative voltage. These signs match the implementation in `set_motor_voltage_no_stick()`.

# 4 Question E

The objective of this question is to determine the equation which will convert the desired angular position into the necessary voltage that is to be sent to the motor. Said equation for a standard DC electric motor is as follows, with the full derivation in Appendix A:

$$\frac{\Theta(s)}{V(s)} = \frac{K_1}{s(\tau s + 1)} \tag{5}$$

This is a standard position-transfer-function model for a DC motor. The constants $K_1$ and $\tau$ represent values that are different to each motor, including the resistance and inductance of internal components. In order to determine these values, the motor must be experimentally characterized.

At this point in the project, the system exists in an open-loop state, as can be seen in Figure 1, where the plant equation is the transfer function 5 and the controller is a simple proportional value. This system does not include a feedback loop, and therefore is not stable enough to be characterized accurately. By providing unity feedback (via the potentiometer, to feed back the current angular position of the motor) the system becomes closed-loop, as can be seen in Figure 2. and the system can be characterized.

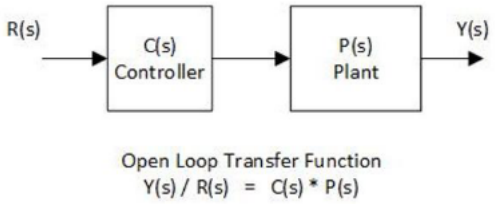



Figure 1: Open Loop System Block Diagram

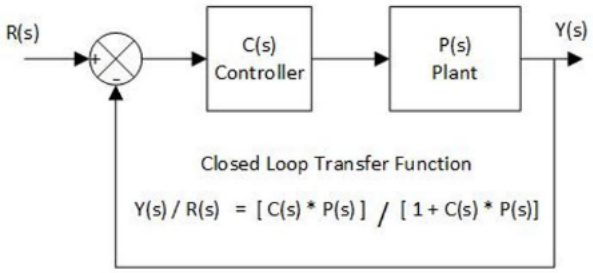



Figure 2: Closed Loop System Block Diagram

Due to the unity feedback, the transfer function changes to the following equation:

$$\frac{\Theta(s)}{\Theta_{ref}(s)} = \frac{\frac{K_p K_1}{s(\tau s+1)}}{1+\frac{K_p K_1}{s(\tau s+1)}} = \frac{\omega_n^2}{s^2+2\zeta\omega_n s+\omega_n^2} \tag{6}$$

The two new unknown values in the above equations, $\omega_n$ and $\zeta$, can be determined through the following equations, which are based on the motor response when following a target theta value:

$$\zeta = \frac{-\ln(\%OS/100)}{\sqrt{\pi^2+[\ln(\%OS/100)]^2}} \tag{7}$$

where $\%OS$ represents overshoot percentage - the percentage that the motor response overshoots the target signal;

$$\omega_n = \frac{\pi}{T_p\sqrt{1-\zeta^2}} \tag{8}$$

where $T_p$ represents the peak time - the time the motor response takes to reach its peak;

$$T_s = \frac{4}{\zeta\omega_n} \tag{9}$$

where $T_s$ represents settling time - the time the motor response takes to reach 2% of the target value.

In order to determine the above values experimentally, a step response was created, and then input into the closed loop system as the target theta value. The error would then be calculated based on the current measured theta value and would be fed to the controller. Multiple values for the proportional coefficient of the controller were used, all of which provided a sufficiently large overshoot to measure; this was done to ensure experimental accuracy. The first value found to display a sufficiently large overshoot was $K_{p_1} = -25$, followed by $K_{p_2} = -28$ and $K_{p_3} = -30$.

While a larger proportional coefficient value would have resulted in a larger overshoot, which would have been easier to study and use in the characterization process of the motor, the maximum value that should have been used in this case is $K_p = -30$. The lab manual has set two values that must be followed: the maximum error for this characterization exercise, which is 0.2 rad, and the maximum allowed voltage to be applied to the motor, which is 6 V. Since the voltage applied to the motor is determined by the controller, which multiplies the maximum error by the proportional coefficient, the largest proportional gain magnitude that would not exceed the maximum voltage is $K_p = -30$. Exceeding this limit would lead to saturation of the motor voltage in the code, which would result in an inaccurate characterization of the motor.

The values of the step response were sampled at a rate of 2000 times per second, or 2 kHz. This value was experimentally determined to be the most ideal for the given situation. The clock of the Arduino R4 Minima runs at 48 MHz, and the ISR was expected to run within 4800 instructions. Given these values, the ideal sampling rate was determined to be 10kHz. While this sampling rate initially worked, noise from the ADC caused many outliers when measuring the motor step response. For this reason, the ADC was to be sampled five times and averaged for each reading. This required the frequency to be divided by five (due to the increased rate of ADC sampling), resulting in a final sampling rate of 2 kHz.

The graphs for the step response of the motor can be seen below, in figures 3, 4, and 5.

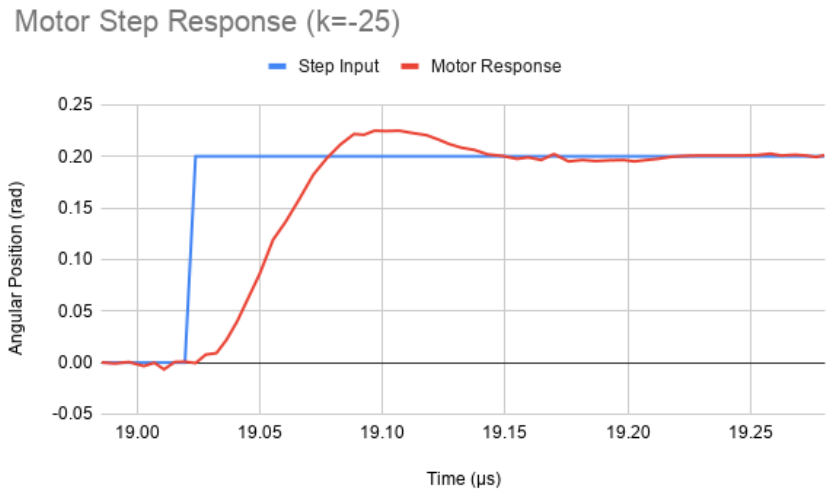



Figure 3: Motor response to step input when $K_p = -25$

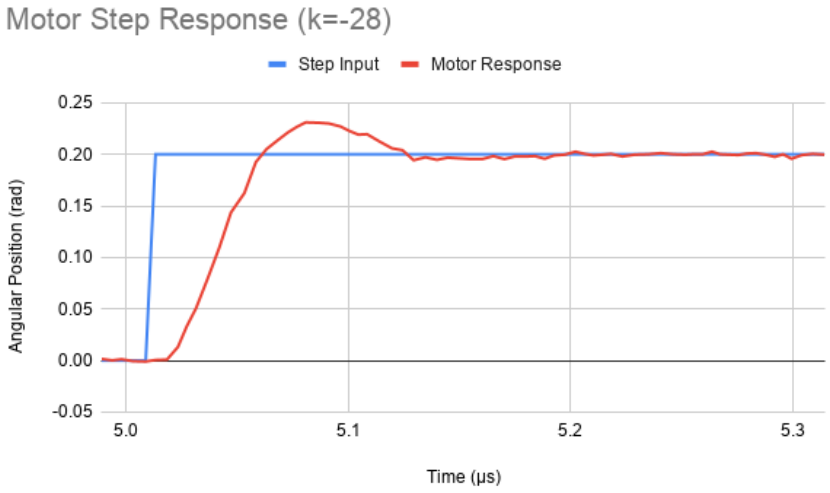



Figure 4: Motor response to step input when $K_p = -28$

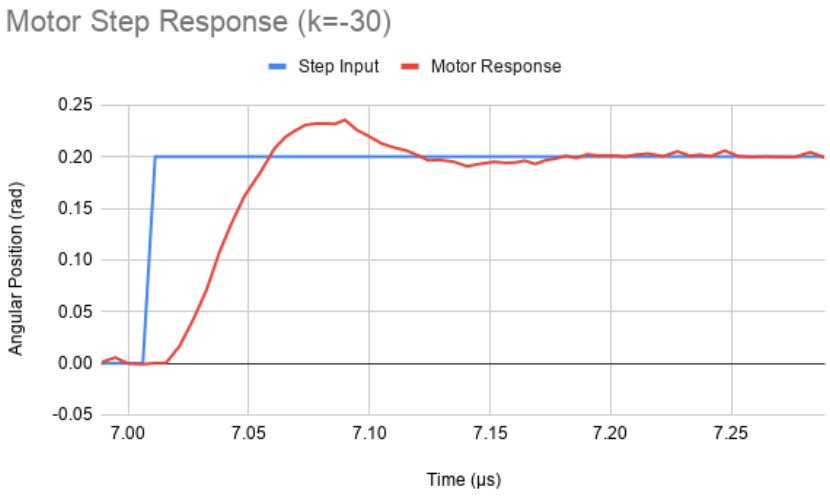



Figure 5: Motor response to step input when $K_p = -30$

The above graphs, which show the motor response to the step input, prove that the stiction values selected in section 3 are adequate. If the stiction value were too low, flat zones in the initial overshoot peak would be present. This is because a deadzone range which is too small would result in certain voltage values that are too small to overcome the motor's static friction, resulting in flat zones in motor response peaks. If the deadzone range was too large, there would be higher steady-state error as well as more oscillations than necessary to reach the steady-state. This is due to the precision that is lost when dealing with larger deadzones; the motor controller is no longer able to make precise movements when the error is low, resulting in oscillations and steady-state error. As can be seen in the graphs, there is neither flat zones nor steady-state error, proving the stiction values are adequate.

From the above graphs, the overshoot percentage and peak time values were determined to be as followed:

$$\%OS_{k=-25} = 12.415\%, T_{p_{k=-25}} = 0.083069s \tag{10}$$

$$\%OS_{k=-28} = 15.47\%, T_{p_{k=-28}} = 0.06758s \tag{11}$$

$$\%OS_{k=-30} = 16.01\%, T_{p_{k=-30}} = 0.066472s \tag{12}$$

Using the formulas 7 and subsequently 8, the following values were able to be formulated:

$$\zeta_{k=-25} = 0.5532, \omega_{n_{k=-25}} = 45.3987 \text{ rad/s} \tag{13}$$

$$\zeta_{k=-28} = 0.5107, \omega_{n_{k=-28}} = 54.0710 \text{ rad/s} \tag{14}$$

$$\zeta_{k=-30} = 0.5037, \omega_{n_{k=-30}} = 54.7104 \text{ rad/s} \tag{15}$$

Comparing the normalized closed-loop denominator with the standard second-order form gives

$$\tau = \frac{1}{2\zeta\omega_n}, \qquad K_1 = \frac{\omega_n^2 \tau}{K_p} = \frac{\omega_n}{2\zeta K_p}.$$

For example, using $K_p = -25$, $\zeta = 0.5532$, and $\omega_n = 45.3987$ rad/s gives

$$\tau = \frac{1}{2(0.5532)(45.3987)} = 0.019909 \text{ s}, \qquad K_1 = \frac{45.3987}{2(0.5532)(-25)} = -1.6413 \text{ rad/(V s)}.$$

Applying the same calculation to the other two data sets gives:

$$K_{1_{k=-25}} = -1.6413 \text{ rad/(V s)}, \quad \tau_{k=-25} = 0.019909 \text{ s} \tag{16}$$

$$K_{1_{k=-28}} = -1.8906 \text{ rad/(V s)}, \quad \tau_{k=-28} = 0.018107 \text{ s} \tag{17}$$

$$K_{1_{k=-30}} = -1.8103 \text{ rad/(V s)}, \quad \tau_{k=-30} = 0.018144 \text{ s} \tag{18}$$

The values for $K_1$ and $\tau$ all fall within the ranges prescribed by the lab manual; with the motor constants properly defined, the motor has successfully been characterized.

# 5 Question F

Choosing $K_p = -28$ as an arbitrary selection, as well as its associated $K_1$ and $\tau$ values, the closed loop system used to characterize the motor was modelled in Simulink software. This was done to ensure that the characterization was done correctly, as accurate values for $K_1$ and $\tau$ would show a motor response which correctly follows $\theta_{desired}$.

A saturator function was also added to the modelled system in order to keep $\theta_{desired}$ within a given range, as in the future lab sessions, the beam will be attached to the motor, and moving outside of the range may result in property damage. This saturator was also tested in the modelled system.

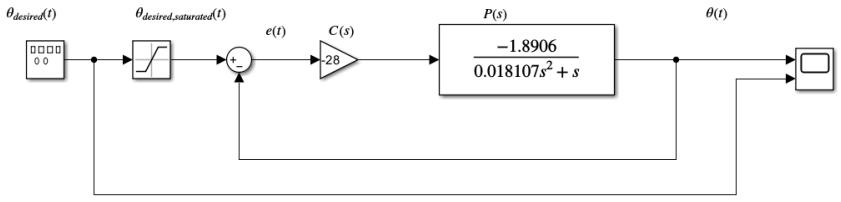



Figure 6: Closed-loop system modelled in Simulink software

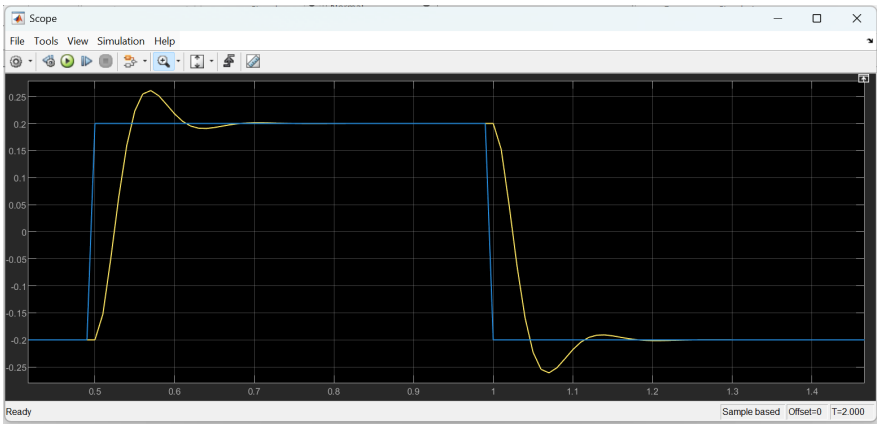



Figure 8: Closed-loop system model: step response

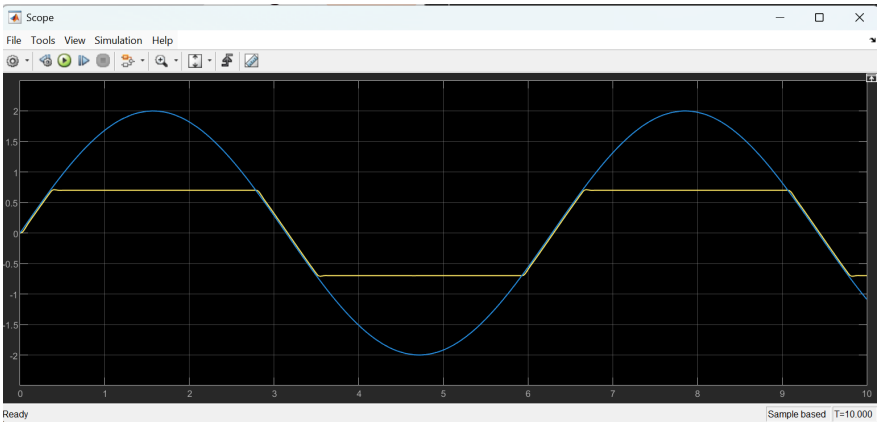



Figure 7: Closed-loop system model output; the blue line represents $\theta_{desired}$ and the yellow line represents $\theta$

The code implementation for the target angular position saturator was straightforward. The experimental saturation test applied both positive and negative reference angles. The measured saturated reference remained within $-0.7$ and $+0.7$ rad, regardless of the unsaturated target value, as shown in Figure 9. The plot includes the unsaturated reference and the saturated reference so that the limiting action can be verified directly.

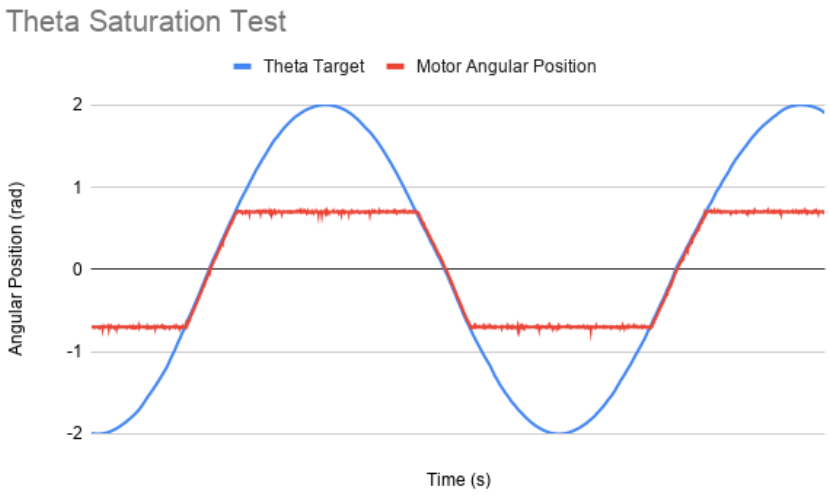



Figure 9: Theta Saturation Test

## 6 Question G

Figure 10 shows the inner loop at the end of Lab 1. The diagram includes the reference saturation, proportional controller, stiction compensation, motor-voltage saturation, PWM and direction output, motor and gear apparatus, potentiometer, ADC averaging, and sensor scaling. Software signals are shown separately from the physical analog signals. The motor, gear, and potentiometer are identified as parts of the apparatus. The controller runs every 500 $\mu$s (2 kHz).

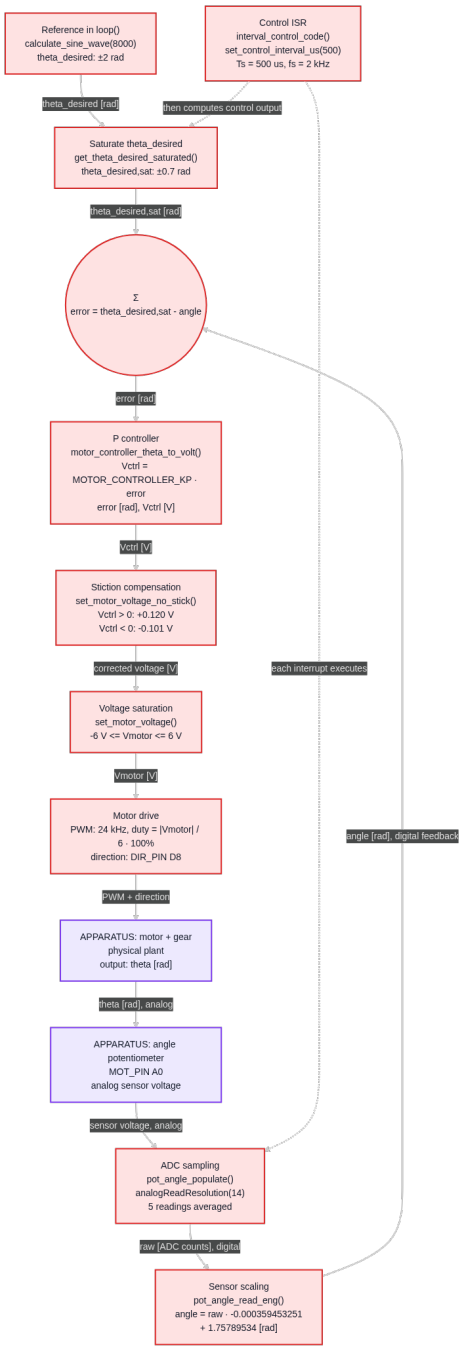



Figure 10: Inner-loop block diagram at the end of Lab 1.

## References


1. S. Livera, *MTE 484: Digital Control Systems — Lab 1: Modelling of the Motor System*, Fall 2026.


# 7 Appendices

## 7.1 Appendix A: Standard DC Electric Motor Transfer Function Derivation

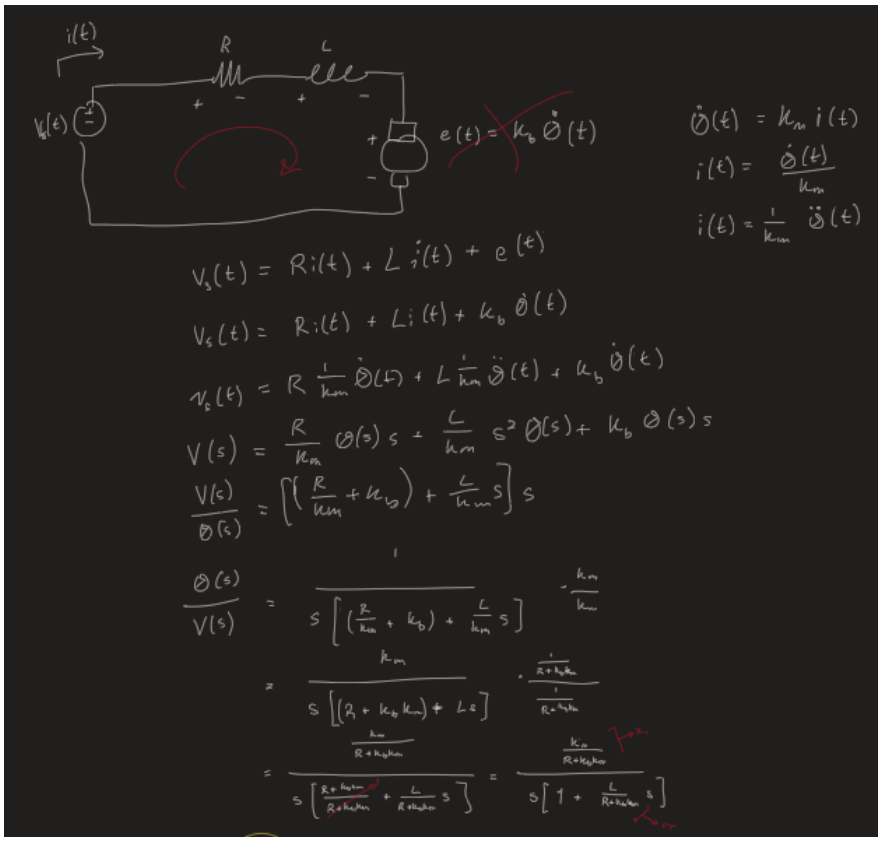



Figure 11: Standard DC Electric Motor Transfer Function Derivation

## 7.2 Appendix B: Code

Listing 1: main.ino

```
#include "src/geeWhiz/geeWhiz.h"
#include "src/motor.h"
#include "src/position.h"
#include "src/transfer.h"
#include <Arduino.h>
#include <cstdint>

#define ISR_PERIOD_US 500
// ================== Pins ==================
```

```
int MOT_PIN = A0; // motor angle sensor
int BAL_PIN = A1; // ball position sensor

int PWM_PIN = D9; // motor PWM
int DIR_PIN = D8; // direction

typedef enum
{
  APP_STATE_CALCULATE_OVERSHOOT,
  APP_STATE_CALCULATE_STICKTION,
  APP_STATE_SINE_WAVE,
} AppState;

static AppState s_app_state;
uint32_t logging_start_us;

void calculate_stiction(void);

// ================== Setup ==================
void setup()
{

  analogReadResolution(14);
  pinMode(A5, OUTPUT); // A5 can be used to measure cycle time using an
      oscilloscope by connecting the scope to the Arduino Box Motor
     Leads
  Serial.begin(115200);

  pot_angle_config(MOT_PIN, 5);
  motor_config(PWM_PIN, DIR_PIN);

  delay(300);

  pot_angle_populate();
  motor_init();
  set_control_interval_us(ISR_PERIOD_US); // 2 kHz

  s_app_state = APP_STATE_SINE_WAVE;
  logging_start_us = micros();
  Serial.println("initialization complete!");
}

// ================== Loop ==================
uint64_t sample = 0;
void loop()
{
  switch (s_app_state)
  {
```

```
  case APP_STATE_CALCULATE_OVERSHOOT:
    calculate_overshoot(1000);
    break;
  case APP_STATE_CALCULATE_STICKTION:
    calculate_stiction();
    break;
  case APP_STATE_SINE_WAVE:
    calculate_sine_wave(8000);
    break;
  }

  float angle = pot_angle_read_eng();
  // uint32_t elapsed_us = micros() - logging_start_us;
  // Serial.print(-0.1);
  // Serial.print(", ");
  // Serial.print(0.3);
  // Serial.print(", ");
  // Serial.print(sample);
  // Serial.print(", ");
  // Serial.print(elapsed_us / 1000000.0f, 6);
  // Serial.print(", ");
  // Serial.print(MOTOR_CONTROLLER_KP);
  // Serial.print(", ");
  // Serial.print(ISR_PERIOD_US);
  // Serial.print(", ");
  Serial.print(get_theta_desired(), 5);
  Serial.print(",");
  Serial.println(angle, 5);
  sample++;
}
// motor voltage: 0.12200007 | theta: -0.13642
// motor voltage: 0.11400005 | theta: -0.21910
// motor voltage: 0.12100007 | theta: -0.28667
// motor voltage: 0.11200005 | theta: -0.35174

// ================== Control ISR ==================
void interval_control_code(void)
{
  pot_angle_populate();
  float angle = pot_angle_read_eng();
  float theta_desired = get_theta_desired_saturated();
  set_motor_voltage_no_stick(motor_controller_theta_to_volt(
     theta_desired, angle));
}
```

Listing 2: motor.h

```
#ifndef MOTOR_H
```

```
#define MOTOR_H

#include <Arduino.h>

void motor_config(int pwm_pin, int dir_pin);

int motor_init();

void set_motor_voltage_no_stick(float volts);

void set_theta_desired(float theta);

float get_theta_desired(void);

float get_theta_desired_saturated(void);

void calculate_overshoot(uint32_t period_ms);

void calculate_sine_wave(uint32_t period_ms);

#endif
```

Listing 3: motor.cpp

```
#include "geeWhiz/geeWhiz.h"
#include "position.h"
#include "transfer.h"
#include <atomic>
#include <cmath>
#include <pwm.h>

#define POS_V_STICTION 0.120f
#define NEG_V_STICTION -0.101f
#define THETA_DESIRED_MAX 0.7f
#define THETA_SWEEP_AMPLITUDE 2.0f

static int s_motor_dir_pin;
static bool s_motor_dir_pin_configured = false;
static PwmOut s_motor_pwm(0);
static bool s_motor_pwm_configured = false;
static constexpr float PWM_HZ = 24000.0f;
static constexpr float VMAX = 6.0f;
static constexpr float DUTY_MAX = 99.2f;
static std::atomic<float> s_theta_desired{0.0f};

// must be called before motor init
void motor_config(int pwm_pin, int dir_pin)
{
```

```
  PwmOut configured_pwm(pwm_pin);
  s_motor_pwm = configured_pwm;
  s_motor_pwm_configured = true;

  s_motor_dir_pin = dir_pin;
  s_motor_dir_pin_configured = true;
}

int motor_init()
{
  if (!s_motor_pwm_configured || !s_motor_dir_pin_configured)
  {
    return -1;
  }

  pinMode(s_motor_dir_pin, OUTPUT);
  digitalWrite(s_motor_dir_pin, LOW);

  s_motor_pwm.begin(PWM_HZ, 0.0f);
  return 0;
}

static void set_motor_voltage(float volts)
{
  if (volts > VMAX)
    volts = VMAX;
  if (volts < -VMAX)
    volts = -VMAX;

  digitalWrite(s_motor_dir_pin, (volts >= 0.0f) ? HIGH : LOW);
  float duty = (fabsf(volts) / VMAX) * 100.0f;
  if (duty > DUTY_MAX)
    duty = DUTY_MAX;

  s_motor_pwm.pulse_perc(duty);
}

void set_motor_voltage_no_stick(float volts)
{
  if (volts > 0.0f)
  {
    set_motor_voltage(volts + POS_V_STICTION);
  }
  else if (volts < 0.0f)
  {
    set_motor_voltage(volts + NEG_V_STICTION);
  }
  else
```

```
  {
    set_motor_voltage(0);
  }
}

static float saturate_theta_desired(float theta_desired)
{
  if (theta_desired > THETA_DESIRED_MAX)
  {
    return THETA_DESIRED_MAX;
  }
  else if (theta_desired < -THETA_DESIRED_MAX)
  {
    return -THETA_DESIRED_MAX;
  }
  else
  {
    return theta_desired;
  }
}

void set_theta_desired(float theta)
{
  s_theta_desired.store(theta, std::memory_order_relaxed);
}

float get_theta_desired(void)
{
  return s_theta_desired.load(std::memory_order_relaxed);
}

float get_theta_desired_saturated(void)
{
  return saturate_theta_desired(get_theta_desired());
}

void calculate_overshoot(uint32_t period_ms)
{
  float theta0 = 0.0f;
  float theta1 = 0.2f;

  static uint32_t t0 = millis();
  static bool state = false;

  if (millis() - t0 > period_ms)
  {
    state = !state;
    t0 = millis();
```

```
  }

  float theta_target = state ? theta1 : theta0;
  set_theta_desired(theta_target);
}

void calculate_sine_wave(uint32_t period_ms)
{
  if (period_ms == 0)
  {
    set_theta_desired(0.0f);
    return;
  }

  float cycle_fraction = static_cast<float>(millis() % period_ms) /
                         static_cast<float>(period_ms);
  float phase = 2.0f * PI * cycle_fraction;
  set_theta_desired(THETA_SWEEP_AMPLITUDE * sinf(phase));
}

/// @brief Calculate and print stiction of motor, disables interrupts
void calculate_stiction(void)
{
  noInterrupts();
  pot_angle_populate();
  float initial_theta = pot_angle_read_eng();
  float theta = initial_theta;
  float motor_voltage = 0.0;
  while (abs(theta - initial_theta) < 0.06)
  {
    set_motor_voltage(motor_voltage);
    delay(200);
    pot_angle_populate();
    theta = pot_angle_read_eng();

    Serial.print("motor voltage:");
    Serial.print(motor_voltage, 8);
    Serial.print("|theta:");
    Serial.println(theta, 5);

    motor_voltage -= 0.001;
  }
  set_motor_voltage(0.0);
  Serial.println("stiction calculation complete... delaying for 10
     seconds");
  delay(1000 * 10);
  interrupts();
}
```

Listing 4: position.h

```h
#ifndef POSITION_H
#define POSITION_H

void pot_angle_config(int angle_pin, int samples);

void pot_angle_populate(void);

int pot_angle_read_raw(void);

float pot_angle_read_eng(void);

#endif
```

Listing 5: position.cpp

```cpp
#include "position.h"
#include "transfer.h"
#include <analog.h>
#include <atomic>

static int s_angle_pin = 0;
static int s_angle_samples = 1;
static std::atomic<int> s_angle_raw{0};

void pot_angle_config(int angle_pin, int samples) {
  s_angle_pin = angle_pin;
  s_angle_samples = samples > 0 ? samples : 1;
}

void pot_angle_populate(void) {
  int sum = 0;
  for (int i = 0; i < s_angle_samples; i++) {
    sum += analogRead(s_angle_pin);
  }

  int raw = (sum + s_angle_samples / 2) / s_angle_samples;
  s_angle_raw.store(raw, std::memory_order_relaxed);
}

int pot_angle_read_raw(void)
{
  return s_angle_raw.load(std::memory_order_relaxed);
}

float pot_angle_read_eng(void)
```

```
{
  return pot_angle_raw_to_rad(pot_angle_read_raw());
}
```

Listing 6: transfer.h

```
#ifndef TRANSFER_H
#define TRANSFER_H

#define MOTOR_CONTROLLER_KP -28

float pot_angle_raw_to_rad(int v);

float motor_controller_theta_to_volt(float theta_target, float theta);

#endif
```

Listing 7: transfer.cpp

```
#include "transfer.h"

#define ANGLE_SLOPE -0.000359453251f
#define ANGLE_INTERCEPT 1.75789534f

float pot_angle_raw_to_rad(int v)
{
    return (float)v * ANGLE_SLOPE + ANGLE_INTERCEPT;
}

float motor_controller_theta_to_volt(float theta_target, float theta)
{
    float err = theta_target - theta;
    return err * MOTOR_CONTROLLER_KP;
}
```

Listing 8: geeWhiz.h

```
#pragma once
#include <Arduino.h>

// Start a periodic ISR at 'interval_us' using a GPT timer (no PWM
   conflict)
void set_control_interval_us(uint32_t interval_us);

// Your sketch must define ONE of these names (either is accepted)
extern "C" void interval_control_code(void) __attribute__((weak));
extern "C" void intervalControlCode(void) __attribute__((weak));
```

Listing 9: geeWhiz.cpp

```
#include "geeWhiz.h"
#include <FspTimer.h> // UNO R4 timer helper

static FspTimer controlTimer; // separate GPT for the control ISR

// ---- ISR adapter ----
static void timer_cb_adapter(timer_callback_args_t *)
{
  if (interval_control_code)
  {
    interval_control_code();
    return;
  }
  if (intervalControlCode)
  {
    intervalControlCode();
    return;
  }
  // neither defined -> do nothing
}

// ---- API ----
void set_control_interval_us(uint32_t interval_us)
{
  if (interval_us == 0)
    interval_us = 1000;
  const float freq_hz = 1000000.0f / interval_us;

  uint8_t timer_type = GPT_TIMER;
  int8_t tindex = FspTimer::get_available_timer(timer_type); // prefer
     non-PWM GPT

  bool forced = false;
  if (tindex < 0)
  {
    // As a fallback, allow using a PWM-reserved GPT if needed
    tindex = FspTimer::get_available_timer(timer_type, true);
    forced = (tindex >= 0);
  }
  if (tindex < 0)
    return; // no timer available

  if (forced)
  {
    // Permit using a PWM-reserved GPT for the timer.
    FspTimer::force_use_of_pwm_reserved_timer();
```

```
  }

  // Begin periodic timer -> IRQ -> open -> start (open is REQUIRED)
  if (!controlTimer.begin(TIMER_MODE_PERIODIC, timer_type, tindex,
      freq_hz, 0.0f, timer_cb_adapter))
    return;
  if (!controlTimer.setup_overflow_irq())
    return;
  if (!controlTimer.open())
    return;
  controlTimer.start();
}
```